

Front End for the Agent

A standalone, deep-dive reference for Build 6 — a Streamlit chat interface wrapping the existing tool-use agent, so the whole project is demoable in a browser instead of only via a Python REPL. Every phase, every real bug the process surfaced, and every verification result, in full. Complements (does not replace) the living cheat sheet and the per-phase flow diagrams.

STATUS

Functionally complete — Phases 1–4 done; Phase 3.5 scoped and deferred (see Known Limitations)

COMMITTS

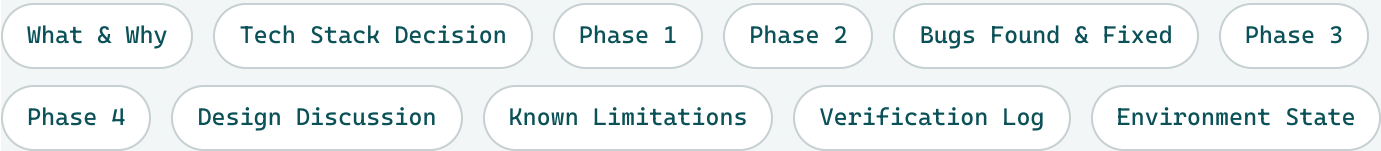
b902500 → 9f82642

CONTINUES

Build 5 (live deployment with CI)

PRECEDES

Build 7 (verifier/critic agent)



What Was Built, and Why

Every build through Build 5 deepened the agent or its infrastructure; none of them changed how a person actually reaches it. `agent.py` was still a `print()`-based CLI loop — technically complete, entirely undemoable to anyone unwilling to clone the repo and run Python locally. Build 6 closes that gap: a real, clickable chat interface, wrapping `ask_agent()` as-is, with no changes to the agent's own reasoning.

Four phases, in order: a minimal chat UI (Phase 1); genuine multi-turn conversation memory, since nothing before this build ever needed it (Phase 2); containerizing the whole thing as a second service sharing Build 1's own Docker image (Phase 3); and a stretch phase surfacing Build 4's tool-call logging directly in the browser (Phase 4). A fifth unit of scope — persistent, restart-proof chat history — was identified mid-build from a real live-testing gap, designed in full, and deliberately deferred; it's documented under Known Limitations as a recorded possible feature, not silently dropped.

Nothing about the agent's own reasoning changed in this build. Every phase here is front-end/infrastructure work around the existing `ask_agent()` tool-use loop — the one exception is a handful of real, evidence-based fixes to `tools.py/agent.py` that live-testing through the new UI surfaced (a missing result cap, stale discovery scope, a stale system prompt) — see Bugs Found & Fixed.

Tech Stack Decision

Decided before any code was written. Streamlit, over two real alternatives.

OPTION	VERDICT
Streamlit (chosen)	Pure Python – no new language or build tooling. Built-in chat primitives (st.chat_message, st.chat_input) purpose-built for exactly this shape of app. Deployable as another docker-compose.yml service, or free via Streamlit Community Cloud.
Flask/FastAPI + a separate JS/React front end	Deferred – more production-shaped in theory, but real scope creep: a second codebase and build-tooling stack for a project whose point is the SQL/data-agent work, not front-end engineering.
Gradio	Deferred – comparable fit, more ML-demo-oriented. Streamlit's broader general adoption and pure-Python fit won out.

1 Minimal Chat UI

Goal: wrap the existing `ask_agent()` loop in a real chat interface — nothing more.

`app.py` – the shape of every later phase builds on

```
import streamlit as st
from agent import ask_agent

# Streamlit reruns this whole script on every interaction, so history has to live in
# session_state explicitly - a plain Python list would reset to empty on every rerun.
if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

user_question = st.chat_input("Ask about the database...")
if user_question:
    # append user message, call ask_agent(), append assistant message
```

Why `agent.py` needed a refactor first, not just `app.py`. The original agent was a `__main__`-guarded script, not an importable function. Build 4 had already refactored it into a callable `ask_agent(user_question)` for the eval harness's benefit — this build is the second, independent payoff of that same refactor, reusing it entirely unchanged rather than duplicating agent logic inside the UI layer.

2 Hybrid Tiered Conversation Memory

Goal: real multi-turn memory. Phase 1's `session_state.messages` only gave the *display* history — `ask_agent()` itself still built a fresh one-message list on every call, so the model had zero actual memory of prior turns despite the transcript rendering fine.

Two designs were weighed, not defaulted into. Option A: full raw Anthropic message-block replay (real tool-call memory, but unbounded per-conversation growth). Option B: text-only history (just prior question/answer text pairs). B was initially recommended as the better-scoped fit — then explicitly overridden once the project's own "production-grade portfolio" goal was invoked: real production systems bound/manage history rather than replay it forever (this assistant's own context-compaction being the concrete example), which argues for *B plus an explicit cap*, not for unbounded A. The chosen design went further still, deliberately: **hybrid tiered memory** — recent rounds keep full raw tool-call fidelity, older rounds collapse to plain text.

agent.py — the actual mechanism

```
MAX_FULL_FIDELITY_ROUNDS = 3
```

```
def flatten_history(history_rounds):
    older_rounds = history_rounds[:-MAX_FULL_FIDELITY_ROUNDS]
    recent_rounds = history_rounds[-MAX_FULL_FIDELITY_ROUNDS:]

    messages = []
    for round in older_rounds:
        messages.append({"role": "user", "content": round["user_question"]})
        messages.append({"role": "assistant", "content": round["answer_text"]})
    for round in recent_rounds:
        messages.extend(round["full_messages"])
    return messages

def ask_agent(user_question, max_tool_turns=MAX_TOOL_TURNS, history_rounds=None):
    messages = flatten_history(history_rounds or [])
    messages.append({"role": "user", "content": user_question})
    # ...turn loop unchanged... every return now also hands back full_messages
```

A genuine cross-layer nuance, corrected while designing: Streamlit's `session_state` keeps live Python objects in server memory across reruns within one session — it does *not* require JSON serialization the way real cross-process persistence would. Storing raw Anthropic SDK objects (`TextBlock/ToolUseBlock`) in `session_state` is simpler than it first sounds; the real complexity is the flatten/collapse logic itself and the API's strict `tool_use`→`tool_result` adjacency rule at the fidelity-window boundary.

NOTE-WORTHY — a performance/best-practice analysis was requested and given before any code was written, flagged explicitly for this brief.

Scale tradeoffs identified, none yet mitigated in the design as built:

1. The fidelity window bounds *round count*, not *per-round payload size* — a much larger table means a single retained `run_sql_query` result could be large regardless of how few rounds are kept raw. (This specific gap became real days later — see Bugs Found & Fixed.)
2. A sliding full→collapsed boundary invalidates the API's prompt-cache prefix every turn (the boundary round's content changes each time) — unlike infrequent/periodic compaction (this assistant's own context management, LangChain's `ConversationSummaryBufferMemory`), which preserves a stable cacheable prefix far longer.
3. Vector-search latency (`search_summaries/search_docs`) compounds with conversation-memory latency at real scale.
4. Per-session server memory footprint ($N \text{ full rounds} \times \text{concurrent users}$, once this build containerizes the app) is real concurrency territory — the same category Build 4's `question_id` field exists for.

Best-practice verdict, precise, not a blanket yes/no: the general shape (recent-full + older-degraded) is a real, recognized pattern — it matches LangChain's summary-buffer memory and this assistant's own context compaction. But this project's version is a simplification in two specific ways: it retains old rounds' verbatim text rather than re-summarizing via a dedicated LLM call, and it's recency-based rather than relevance-based (semantic retrieval of the most pertinent prior rounds) — this codebase is unusually well-positioned to do the latter, since `pgvector` is already proven via `search_summaries/search_docs`. A second, more sophisticated alternative already sits in this codebase too: push large tool outputs into `agent_scratch` instead of retaining them in context, and have the agent re-query rather than re-read.

Disposition: named honestly as future-examination material, not blocking — Phase 2 proceeded on the hybrid design as scoped.

Step 8 — the boundary test, confirmed genuinely, not assumed. Two test designs were tried and honestly discarded before a third worked: the first two were inconclusive because the seed question's own answer was already a complete data dump, so collapsed text alone would have sufficed regardless of whether true collapse was actually exercised. The third design deliberately asked only for an aggregate total up front, forcing the later follow-up to require fresh tool calls if memory genuinely didn't retain the detail.

Sample — the confirming 3-question chain, `MAX_FULL_FIDELITY_ROUNDS` temporarily set to 1

Q1: "What's the average allocation duration, in hours?" # aggregate only, no row detail
-> 230.06 hours

Q2: "Which allocation had the most items checked out?" # unrelated, just advances the round count
-> allocation 9993490, 95 items

Q3: "What were the exact items in that allocation?"

Q2 is now collapsed, not full-

fidelity

-> correctly re-ran run_sql_query rather than fabricating an answer

-> allocation 9993490, 6 items # (reconstructed correctly - re-queried, not hallucinated)

Reproduced identically across two script-based runs and one real browser session — the system recovers gracefully via re-querying when retained memory doesn't have what's needed, arguably a more important property than "recalls old raw data verbatim."

Bugs Found & Fixed

All five surfaced through genuine live-testing of the new UI, not speculative review — the first real end-to-end usage this agent had ever gotten outside a controlled script or eval case.

1. `run_sql_query` had no result-size cap — broke a live browser question with a real 400 API error before Phase 2's own code was even finished. An unfiltered follow-up-style question (`SELECT ... FROM clean.allocations WHERE actual_end IS NOT NULL ORDER BY actual_end DESC`, no `LIMIT`) matched nearly the whole table and returned 936,551 characters in one tool result — Error code: 400 ... prompt is too long: 448866 tokens > 200000 maximum. Every prior use of this tool (Build 1 through the eval harness) happened to involve small tables, aggregates, or model-written `LIMIT`s, so this path had simply never been exercised before.

Fixed: `MAX_SQL_RESULT_ROWS = 200`, `cur.fetchmany(201)` instead of `fetchall()` — fetching one row past the cap makes truncation detectable without a second round-trip. A truncated result gets a trailing `{"message": "Result truncated at 200 rows..."}` sentinel entry, the same pattern `search_summaries/search_docs` already use for "no match." This exact gap had already been named as a real, unaddressed risk during Phase 2's own scale discussion above — this incident is that flagged risk surfacing for real, independently confirming the flag wasn't paranoid.

2. The tool-call log never recorded the actual question asked. `logs/tool_calls.jsonl` had every field describing what a tool did, but not what the user had asked that triggered it — discovered while trying to hand back the exact original question that had caused bug 1, and finding it structurally impossible to reconstruct from the log alone.

Fixed: `log_tool_call()` gained a `user_question` parameter, now passed from both of `ask_agent()`'s call sites.

3. `docs.chunks` — the documentation-RAG corpus — had silently stopped growing after Build 3. Found by directly noticing that questions about build history only ever surfaced Build 1–3 content. Root cause: `extract_doc_chunks.py`'s `SOURCE_FILES` is a hardcoded list, written during Build 3.5 and never updated as Build 3.5, 4, and 5 each got their own handoff brief afterward.

Fixed: the three missing brief paths added, extraction/embedding scripts re-run — both already idempotent by design (`DELETE ... WHERE source_file = %s` before re-insert; `WHERE embedding IS NULL` backfill), so

nothing needed to change to make the re-run safe. All 10 source files now present, 395 chunks, 0 unembedded.

4. The log recorded tool activity but never the model's actual final answer. Made correctness impossible to verify from the log alone — had to cross-check every answer against the live browser by hand.

Fixed: a new `log_final_answer(question_id, user_question, answer, error)` event — a distinct shape from tool-call entries (no `tool_name/turn/input/output/latency_ms`), wired into all three of `ask_agent()`'s return points.

5. The system prompt still described appdb as a generic "small business database," leftover Build 1 framing. Asking "What can you do?" through the real UI produced example questions like "customers with orders over \$500" — confirmed via the log that zero tool calls fired, meaning this was pure stale-framing filler text, not real schema discovery. A real portfolio viewer trying one of those self-suggested examples would have gotten a confusing empty result, since those tables were never the project's actual subject.

Fixed, alongside related housekeeping: the prompt's opening rewritten to describe the actual domain (equipment allocations, `clean.allocations` as the real subject table). Separately, `customers/orders` (original Build 1 seed tables, unrelated to the portfolio narrative since Build 2) removed from `list_tables/describe_table`'s discovery scope — a discovery-level fix, not a hard DB-level lockout, since this isn't sensitive data, just narrative clutter.

3 Containerization

Goal: run the Streamlit app as a real docker-compose service, matching how the CLI agent has run since Build 1.

A real pre-existing bug found before any Phase 3 code was written. Dockerfile's COPY agent.py tools.py ./ line had never included logging_utils.py (added in Build 4) — the agent service's image would have failed to build with an ImportError the moment it was ever rebuilt, unrelated to Phase 3 itself but only caught because this phase meant touching this exact file.

Fixed: COPY agent.py tools.py logging_utils.py app.py ./.

docker-compose.yml – one image, two services, differentiated only by command

```
agent:
  build: .
  # default CMD ["python", "agent.py"] from the Dockerfile

streamlit_app:
  build: .
  depends_on: [postgres]
  ports:
    - "8501:8501"
  volumes:
    - ./logs:/app/logs
  command: ["streamlit", "run", "app.py", "--server.address=0.0.0.0"]
```

A genuine Streamlit-in-Docker gotcha: Streamlit defaults to binding localhost only — unreachable through Docker's port mapping without --server.address=0.0.0.0, which was not obvious from the plain streamlit run command used locally in Phase 1.

Stricter than the local dev-server case: Docker's COPY bakes source in at build time, with no hot-reload path at all. Editing agent.py/tools.py/app.py on the host has zero effect on an already-running container. Confirmed directly (not assumed) after the Bug 5 system-prompt fix: a docker exec ... grep against the running container's own copy of agent.py showed the old text still present until a full docker compose build + up (recreate) was run.

4 Tools Used Expander

Goal (stretch): surface Build 4's existing tool-call logging directly in the browser, rather than leaving it CLI/log-file-only.

Design: reuse the log as the source of truth, don't track calls twice. Matches the precedent already set by `run_eval.py` (which reads `logs/tool_calls.jsonl` after each `ask_agent()` call rather than tracking separately).

`logging_utils.py` – the one new function this phase needed

```
def get_tool_calls(question_id):
    if not os.path.exists(LOG_PATH):
        return []
    calls = []
    with open(LOG_PATH) as f:
        for line in f:
            entry = json.loads(line)
            if entry.get("question_id") == question_id and "tool_name" in entry:
                calls.append({"tool_name": entry["tool_name"], "input": entry["input"],
                    "latency_ms": entry["latency_ms"]})
    return calls
```

`ask_agent()`'s three return points now hand back `question_id` (already generated internally, just never returned before); `app.py` calls `get_tool_calls(question_id)` after each response and renders an `st.expander`, storing the result per round in `session_state` so it survives future reruns instead of only showing once.

Deliberate scope limit: input only, never output. A tool's output can be a full row dump (the exact shape Bug 1 above had to be capped for) — showing it in the expander would just move the doom-scrolling problem into a different widget. Tool name, input, and latency (as a small caption) are shown; the underlying data is not.

A real follow-up caught live, not during initial build: the expander was originally hidden entirely when a question triggered zero tool calls (e.g. "What can you do?", answered purely from the system prompt). Confirmed via the log this was correct behavior, not a bug — then changed anyway, per explicit request, to always render `Tools used (0)` with a short explanatory caption, so the UI element is consistently present rather than silently absent.

Design Discussion

One Image, Two Containers

Two services, two separate needs, not a redundancy. `agent` (the CLI container) is the original Build 1 deliverable, proving the core tool-use loop works, containerized and reproducible — also what Build 5's CI actually exercises (migrations + `pytest` run against the real agent code), the engineering foundation everything else sits on. `streamlit_app` (the web container) was built specifically because the CLI isn't demoable to a portfolio reviewer — per Build 6's own original scoping: a real, clickable interface so the project is demoable without cloning the repo and running Python locally.

Both containers are built from the **same image** (One `Dockerfile`, One `requirements.txt`, one set of copied source files) but run with **different commands** — not two separate images, not one container doing double duty. If asked "which one is the final project": the Streamlit container is what a recruiter would actually click through, but the CLI/agent container isn't superseded by it — it's the tested, CI-verified core the Streamlit app is a front end on top of. Both stay in the final project, serving different audiences.

What Is `sentence-transformers` Actually Doing?

Not solely focused on the allocations data — it plays the same *mechanical* role in two entirely separate places. It converts text into a fixed-length numeric vector (384 dimensions, `all-MiniLM-L6-v2`) — nothing more. It never writes any part of the agent's actual response; that's entirely Claude's job. The embedding model's only output is "here's a vector representing the meaning of this text," compared to pre-computed vectors via distance (`<=>` in `pgvector`) to find semantically similar rows.

Two distinct uses in `tools.py`, sharing one cached model instance: `search_summaries()` embeds the user's query and compares it against `clean.allocations.summary_embedding` — the allocations data itself. `search_docs()` embeds the query and compares it against `docs.chunks.embedding` — this project's own build history and design decisions, nothing to do with the allocations database at all.

The actual flow, end to end: the embedding model finds candidate rows semantically close to the question. Those rows return to Claude as a normal tool result, the same way `run_sql_query`'s rows would — and Claude is the one that reads them and writes the actual prose answer. `sentence-transformers` never sees the conversation, never generates text for the user, and has no awareness of which tool called it.

Known Limitations & Open Items

Chat history is not persistent — a container restart silently wipes every session's memory. Found live: a restart (done to pick up the Bug 5 prompt fix) wiped an in-progress browser conversation with no in-UI signal it had happened. Confirmed via a direct docker exec test that the memory mechanism itself works correctly inside Docker (a fresh session recalling a prior answer with zero new tool calls) — the real cause is that `st.session_state` lives only in server memory, with nothing persisting it. A `PROCESS_ID` diagnostic (generated once per process, shown as a UI caption) was added so a restart is at least visible rather than silently confusing.

Scoped in full, deliberately deferred, recorded as a possible future feature — Phase 3.5, persistent shared chat history. Design: a new `agent_scratch.chat_rounds` table (reusing the existing `appdb_agent_writer` role, not a new one), loaded on session start instead of starting empty — `flatten_history()` itself needs zero changes, since persistence only changes where the initial rounds list comes from. Chosen shape: **one shared/global conversation** (not per-session/per-user), explicitly simpler than giving each browser tab a durable identity via `st.query_params`, appropriate for a solo portfolio demo. Two real design gaps were identified during scoping and left undecided, since the feature itself is on hold: (1) the shared-feed design has no user/session attribution at all — every tab renders the exact same transcript, so there's no "these are mine" concept; (2) persistence removes the accidental page-length cap a restart used to provide — nothing currently limits how long the rendered UI history grows, which would need an explicit display-only cap (separate from the model's own context-window handling, which is already solved).

Cosmetic-only: Streamlit's dev-mode file watcher logs a harmless `ModuleNotFoundError: No module named 'torchvision'`. Root-caused to the watcher probing optional vision-model submodules while walking sentence-transformers's dependency tree — this project never installs or needs torchvision. Zero functional impact, confirmed every time it came up; a `[logger] level = "error"` config fix was attempted and could not be confirmed effective in real use, so it was removed rather than kept as unverified.

Verification Log

What was actually run against a real environment or a real browser, not just read for plausibility.

Phase 1

Live-verified in the browser; independently rebuilt in place later as a comprehension check before Build 6 was considered closed.

Phase 2, boundary test

Reproduced identically across two script-based runs and one real browser session (see the sample transcript above).

Each of the 5 bugs above

py_compile on every changed file; a direct functional reproduction of the original failure against the fixed code; full pytest suite re-run after every single fix (19 → 22 passing across the build).

Phase 3

Both images built, stack brought up, live-verified in the browser with log persistence confirmed via the host filesystem bind mount; a stale unrelated container from an earlier build cleaned up so only the current one remains.

Phase 4

py_compile clean; a direct functional test of `get_tool_calls` (temp log file — correct `question_id` filtering, output excluded, unknown-ID returns `[]`); full pytest suite 22/22; a real live `ask_agent()` call confirming `question_id` threads end-to-end to real logged data; confirmed live in the browser for both the zero-tool-call and multi-tool-call cases.

Sample – live browser confirmation, Phase 4's zero-tool-call case

Q: "What can you do?"

-> answered entirely from the system prompt, zero tool calls

-> Tools used (0)

"Answered directly - no tools were called for this question."

Q: "Tell me about the database?"

-> list_tables {} 22ms

-> describe_table {"table_name": "allocations"} 20ms

-> Tools used (2), both persisted correctly after the next question was asked

Environment State

COMPONENT	DETAIL
New Python	app.py – the Streamlit endpoint
Changed Python	agent.py (ask_agent() callable refactor carried in from Build 4, flatten_history(), MAX_FULL_FIDELITY_ROUNDS, PROCESS_ID, question_id returned, SYSTEM_PROMPT rewritten), tools.py (MAX_SQL_RESULT_ROWS cap, customers/orders removed from discovery), logging_utils.py (user_question field, log_final_answer(), get_tool_calls())
Changed infra	Dockerfile (COPY fixed to include logging_utils.py/app.py), docker-compose.yml (new streamlit_app service), requirements.txt (streamlit added)
New tests	test_agent_history.py – 3 tests for flatten_history()
Diagram folder	PROJECT_DIAGRAMS/BUILD_6_FLOW/ – phase1/phase2/phase3/phase3-5/phase4 flow diagrams, this brief. Phase 3.5's diagram documents a deferred design, not built code.
Live app	http://localhost:8501 (streamlit_app service) – http://localhost:8501 unreachable without --server.address=0.0.0.0, see Phase 3
Project test suite	22/22 passing (up from 19 at the start of this build)

Build 6 functionally complete — Phases 1 through 4 done, live-verified in a real browser. Phase 3.5 (persistent shared chat history) is fully designed and deliberately deferred, recorded above as a possible future feature rather than blocking scope. Next: Build 7 (a verifier/critic agent), or Phase 3.5 if it's picked back up first. Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.